

Title

Helmet Detection using YOLOv5

Author:

Sayyed Riza Jahan

Affiliation:

BCA Student | Machine Learning Intern | Mintways

ABSTRACT

In today's world, road safety has become a global concern, especially when it comes to ensuring the use of helmets by two-wheeler riders. The rise in road accidents involving motorcyclists who do not wear helmets has prompted the need for automated helmet detection systems. This project presents a real-time helmet detection model developed using YOLOv5, a highly efficient object detection algorithm, trained on a custom dataset. The system is capable of identifying whether a person is wearing a helmet or not, both from test images and live webcam feeds.

The final solution includes a Streamlit-based web app, allowing users to upload images for detection without the need for technical expertise. The model was trained for 50 epochs, fine-tuned using a dataset structured with proper annotations, and then tested across multiple inputs. This paper outlines the complete development lifecycle—from dataset preparation, training, testing, and evaluation, to creating a user-friendly interface. It demonstrates that with proper optimization, such models can be used effectively in real-world safety and surveillance systems.

INTRODUCTION

Road traffic injuries are a leading cause of death globally. A major contributor to these injuries is the non-use of safety gear, particularly helmets by two-wheeler riders. Manual enforcement of helmet rules can be ineffective and inconsistent. Thus, automated helmet detection using computer vision offers a promising solution.

With the rapid advancement of deep learning techniques, object detection models have significantly evolved. YOLO (You Only Look Once) is one such real-time object detection system that provides both speed and accuracy. YOLOv5, the latest and most efficient implementation by Ultralytics, offers flexibility, ease of training, and deployment on low-power devices, making it suitable for real-time safety enforcement systems.

In this project, I developed a helmet detection system using YOLOv5, trained on a custom dataset. It was further integrated into a Streamlit web application that allows users to test the model by uploading images or using a live camera feed. This approach ensures accessibility, practical usability, and lays the foundation for future integration into traffic monitoring systems.

DATASET OVERVIEW:

The effectiveness of any machine learning model is heavily dependent on the quality and diversity of its dataset. For this project, a custom dataset was created that includes images of individuals with helmets and without helmets.

Dataset Structure:

- **Images:** Divided into train, valid, and test folders.
- **Annotations:** Provided in YOLO format (.txt files) with corresponding bounding boxes and class labels.
- **Classes:**
 - Class 0: Helmet
 - Class 1: No-Helmet

data.yaml file:

```
C: > Users > hplap > PycharmProjects > pythonProject1 > helmet-detection > ! data.yaml
1  train: ../helmet-detection/train/images
2  val: ../helmet-detection/valid/images
3  test: ../helmet-detection/test/images
4
5  nc: 2
6  names: ['helmet', 'no-helmet']
7
```

By ensuring a well-structured dataset with balanced class representation, the model was able to learn the distinguishing features effectively. The dataset was sourced and augmented to improve generalization and avoid overfitting.

MODEL TRAINING:

Training the YOLOv5 model was a critical step in developing the helmet detection system. I used the YOLOv5s architecture for faster training and sufficient accuracy. The training dataset was split into three folders: train, valid, and test, each containing images and corresponding label files in YOLO format.

Train.py file for training the model:

```
C:\Users\hplap > PycharmProjects > pythonProject1 > yolov5 > train.py
1  import argparse
2  import math
3  import os
4  import random
5  import subprocess
6  import sys
7  import time
8  from copy import deepcopy
9  from datetime import datetime, timedelta
10 from pathlib import Path
11
12 try:
13     import comet_ml # must be imported before torch (if installed)
14 except ImportError:
15     comet_ml = None
16
17 import numpy as np
18 import torch
19 import torch.distributed as dist
20 import torch.nn as nn
21 import yaml
22 from torch.optim import lr_scheduler
23 from tqdm import tqdm
24
25 FILE = Path(__file__).resolve()
26 ROOT = FILE.parents[0] # YOLOv5 root directory
27 if str(ROOT) not in sys.path:
28     sys.path.append(str(ROOT)) # add ROOT to PATH
29 ROOT = Path(os.path.relpath(ROOT, Path.cwd())) # relative
30
31 from ultralytics.utils.patches import torch_load
32
33 import val as validate # for end-of-epoch mAP
34 from models.experimental import attempt_load
35 from models.yolo import Model
36 from utils.autoanchor import check_anchors
37 from utils.autobatch import check_train_batch_size
38
39 from utils.callbacks import Callbacks
40 from utils.dataloaders import create_dataloader
41 from utils.downloads import attempt_download, is_url
42 from utils.general import (
43     LOGGER,
44     TQDM_BAR_FORMAT,
45     check_amp,
46     check_dataset,
47     check_file,
48     check_git_info,
49     check_git_status,
50     check_img_size,
51     check_requirements,
52     check_suffix,
53     check_yaml,
54     colorstr,
55     get_latest_run,
56     increment_path,
57     init_seeds,
58     intersect_dicts,
59     labels_to_class_weights,
60     labels_to_image_weights,
61     methods,
62     one_cycle,
63     print_args,
64     print_mutation,
65     strip_optimizer,
66     yaml_save,
67 )
68 from utils.loggers import LOGGERS, loggers
69 from utils.loggers.comet.comet_utils import check_comet_resume
70 from utils.loss import ComputeLoss
71 from utils.metrics import fitness
72 from utils.plots import plot_evolve
73 from utils.torch_utils import (
74     EarlyStopping,
```

```

75     de_parallel,
76     select_device,
77     smart_DDP,
78     smart_optimizer,
79     smart_resume,
80     torch_distributed_zero_first,
81 )
82
83 LOCAL_RANK = int(os.getenv("LOCAL_RANK", -1)) # https://pytorch.org/docs/stable/elastic/run.html
84 RANK = int(os.getenv("RANK", -1))
85 WORLD_SIZE = int(os.getenv("WORLD_SIZE", 1))
86 GIT_INFO = check_git_info()
87
88
89 def train(hyp, opt, device, callbacks):
90
91     save_dir, epochs, batch_size, weights, single_cls, evolve, data, cfg, resume, noval, nosave, workers, freeze = (
92         Path(opt.save_dir),
93         opt.epochs,
94         opt.batch_size,
95         opt.weights,
96         opt.single_cls,
97         opt.evolve,
98         opt.data,
99         opt.cfg,
100         opt.resume,
101         opt.noval,
102         opt.nosave,
103         opt.workers,
104         opt.freeze,
105     )
106     callbacks.run("on_pretrainRoutine_start")
107
108     # Directories
109     w = save_dir / "weights" # weights dir
110     (w.parent if evolve else w).mkdir(parents=True, exist_ok=True) # make dir
111     last, best = w / "last.pt", w / "best.pt"
112
113     # Hyperparameters
114     if isinstance(hyp, str):
115         with open(hyp, errors="ignore") as f:
116             hyp = yaml.safe_load(f) # load hyps dict
117     LOGGER.info(colorstr("hyperparameters: ") + ", ".join(f"{k}={v}" for k, v in hyp.items()))
118     opt.hyp = hyp.copy() # for saving hyps to checkpoints
119
120     # Save run settings
121     if not evolve:
122         yaml_save(save_dir / "hyp.yaml", hyp)
123         yaml_save(save_dir / "opt.yaml", vars(opt))
124
125     # Loggers
126     data_dict = None
127     if RANK in {-1, 0}:
128         include_loggers = list(LOGGERS)
129         if getattr(opt, "ndjson_console", False):
130             include_loggers.append("ndjson_console")
131         if getattr(opt, "ndjson_file", False):
132             include_loggers.append("ndjson_file")
133
134         loggers = Loggers(
135             save_dir=save_dir,
136             weights=weights,
137             opt=opt,
138             hyp=hyp,
139             logger=LOGGER,
140             include=tuple(include_loggers),
141         )
142
143         # Register actions
144         for k in methods(loggers):
145             callbacks.register_action(k, callback=getattr(loggers, k))
146
147         # Process custom dataset artifact link
148         data_dict = loggers.remote_dataset
149         if resume: # If resuming runs from remote artifact
150             weights, epochs, hyp, batch_size = opt.weights, opt.epochs, opt.hyp, opt.batch_size
151
152     # Config
153     plots = not evolve and not opt.noplots # create plots
154     cuda = device.type != "cpu"
155     init_seeds(opt.seed + 1 + RANK, deterministic=True)
156     with torch_distributed_zero_first(LOCAL_RANK):
157         data_dict = data_dict or check_dataset(data) # check if None
158         train_path, val_path = data_dict["train"], data_dict["val"]
159         nc = 1 if single_cls else int(data_dict["nc"]) # number of classes
160         names = {0: "item"} if single_cls and len(data_dict["names"]) != 1 else data_dict["names"] # class names
161         is_coco = isinstance(val_path, str) and val_path.endswith("coco/val2017.txt") # COCO dataset
162
163     # Model
164     check_suffix(weights, ".pt") # check weights
165     pretrained = weights.endswith(".pt")
166     if pretrained:
167         with torch_distributed_zero_first(LOCAL_RANK):
168             weights = attempt_download(weights) # download if not found locally
169             ckpt = torch_load(weights, map_location="cpu") # load checkpoint to CPU to avoid CUDA memory leak
170             model = Model(cfg or ckpt["model"].yaml, ch=3, nc=nc, anchors=hyp.get("anchors")).to(device) # create
171             exclude = ["anchor"] if (cfg or hyp.get("anchors")) and not resume else [] # exclude keys
172             csd = ckpt["model"].float().state_dict() # checkpoint state dict as FP32
173             csd = intersect_dicts(csd, model.state_dict(), exclude=exclude) # intersect
174             model.load_state_dict(csd, strict=False) # load
175             LOGGER.info(f"Transferred {len(csd)}/{len(model.state_dict())} items from {weights}") # report
176         else:
177             model = Model(cfg, ch=3, nc=nc, anchors=hyp.get("anchors")).to(device) # create
178             amp = check_amp(model) # check AMP
179
180     # Freeze
181     freeze = [f"model.{x}." for x in (freeze if len(freeze) > 1 else range(freeze[0]))] # layers to freeze

```

```

182 for k, v in model.named_parameters():
183     v.requires_grad = True # train all layers
184     if any(x in k for x in freeze):
185         LOGGER.info(f"freezing {k}")
186         v.requires_grad = False
187
188 # Image size
189 gs = max(int(model.stride.max()), 32) # grid size (max stride)
190 imgsz = check_img_size(opt.imgsz, gs, floor=gs * 2) # verify imgsz is gs-multiple
191
192 # Batch size
193 if RANK == -1 and batch_size == -1: # single-GPU only, estimate best batch size
194     batch_size = check_train_batch_size(model, imgsz, amp)
195     loggers.on_params_update({"batch_size": batch_size})
196
197 # Optimizer
198 nbs = 64 # nominal batch size
199 accumulate = max(round(nbs / batch_size), 1) # accumulate loss before optimizing
200 hyp["weight_decay"] *= batch_size * accumulate / nbs # scale weight_decay
201 optimizer = smart_optimizer(model, opt.optimizer, hyp["lr0"], hyp["momentum"], hyp["weight_decay"])
202
203 # Scheduler
204 if opt.cos_lr:
205     lf = one_cycle(1, hyp["lrf"], epochs) # cosine 1->hyp["lrf"]
206 else:
207     def lf(x):
208         return (1 - x / epochs) * (1.0 - hyp["lrf"]) + hyp["lrf"] # linear
209
210 scheduler = lr_scheduler.LambdaLR(optimizer, lr_lambda=lf) # plot_lr_scheduler(optimizer, scheduler, epochs)
211
212 # EMA
213 ema = ModelEMA(model) if RANK in {-1, 0} else None
214
215 # Resume
216 best_fitness, start_epoch = 0.0, 0
217
218 if pretrained:
219     if resume:
220         best_fitness, start_epoch, epochs = smart_resume(ckpt, optimizer, ema, weights, epochs, resume)
221     del ckpt, csd
222
223 # DP mode
224 if cuda and RANK == -1 and torch.cuda.device_count() > 1:
225     LOGGER.warning(
226         "WARNING DP not recommended, use torch.distributed.run for best DDP Multi-GPU results.\n"
227         "See Multi-GPU Tutorial at https://docs.ultralytics.com/yolov5/tutorials/multi\_gpu\_training to get started."
228     )
229     model = torch.nn.DataParallel(model)
230
231 # SyncBatchNorm
232 if opt.sync_bn and cuda and RANK != -1:
233     model = torch.nn.SyncBatchNorm.convert_sync_batchnorm(model).to(device)
234     LOGGER.info("Using SyncBatchNorm()")
235
236 # Trainloader
237 train_loader, dataset = create_dataloader(
238     train_path,
239     imgsz,
240     batch_size // WORLD_SIZE,
241     gs,
242     single_cls,
243     hyp=hyp,
244     augment=True,
245     cache=None if opt.cache == "val" else opt.cache,
246     rect=opt.rect,
247     rank=LOCAL_RANK,
248     workers=workers,
249     image_weights=opt.image_weights,
250     quad=opt.quad,
251     prefix=colorstr("train: "),
252     shuffle=True,
253     seed=opt.seed,
254 )
255 labels = np.concatenate(dataset.labels, 0)
256 mlc = int(labels[:, 0].max()) # max label class
257 assert mlc < nc, f"Label class {mlc} exceeds nc={nc} in {data}. Possible class labels are 0-{nc - 1}"
258
259 # Process 0
260 if RANK in {-1, 0}:
261     val_loader = create_dataloader(
262         val_path,
263         imgsz,
264         batch_size // WORLD_SIZE * 2,
265         gs,
266         single_cls,
267         hyp=hyp,
268         cache=None if noval else opt.cache,
269         rect=True,
270         rank=-1,
271         workers=workers * 2,
272         pad=0.5,
273         prefix=colorstr("val: "),
274     )[0]
275
276     if not resume:
277         if not opt.noautoanchor:
278             check_anchors(dataset, model=model, thr=hyp["anchor_t"], imgsz=imgsz) # run AutoAnchor
279         model.half().float() # pre-reduce anchor precision
280
281     callbacks.run("on_pretrainRoutine_end", labels, names)
282
283 # DDP mode
284 if cuda and RANK != -1:
285     model = smart_DDP(model)
286
287 # Model attributes
288 nl = de_parallel(model).model[-1].nl # number of detection layers (to scale hyps)

```

```

289 hyp["box"] *= 3 / nl # scale to layers
290 hyp["cls"] *= nc / 80 * 3 / nl # scale to classes and layers
291 hyp["obj"] *= (imgsz / 640) ** 2 * 3 / nl # scale to image size and layers
292 hyp["label_smoothing"] = opt.label_smoothing
293 model.nc = nc # attach number of classes to model
294 model.hyp = hyp # attach hyperparameters to model
295 model.class_weights = labels_to_class_weights(dataset.labels, nc).to(device) * nc # attach class weights
296 model.names = names
297
298 # Start training
299 t0 = time.time()
300 nb = len(train_loader) # number of batches
301 nw = max(round(hyp["warmup_epochs"] * nb), 100) # number of warmup iterations, max(3 epochs, 100 iterations)
302 # nw = min(nw, (epochs - start_epoch) / 2 * nb) # limit warmup to < 1/2 of training
303 last_opt_step = -1
304 maps = np.zeros(nc) # mAP per class
305 results = (0, 0, 0, 0, 0, 0, 0) # P, R, mAP@0.5, mAP@0.5-.95, val_loss(box, obj, cls)
306 scheduler.last_epoch = start_epoch - 1 # do not move
307 scaler = torch.cuda.amp.GradScaler(enabled=amp)
308 stopper, stop = EarlyStopping(patience=opt.patience), False
309 compute_loss = ComputeLoss(model) # init loss class
310 callbacks.run("on_train_start")
311 LOGGER.info(
312     f"Image sizes {imgsz} train, {imgsz} val\n"
313     f"Using {train_loader.num_workers * WORLD_SIZE} dataloader workers\n"
314     f"Logging results to {colorstr('bold', save_dir)}\n"
315     f"Starting training for {epochs} epochs..."
316 )
317 for epoch in range(start_epoch, epochs): # epoch
318     callbacks.run("on_train_epoch_start")
319     model.train()
320
321     # Update image weights (optional, single-GPU only)
322     if opt.image_weights:
323         cw = model.class_weights.cpu().numpy() * (1 - maps) ** 2 / nc # class weights
324         iw = labels_to_image_weights(dataset.labels, nc=nc, class_weights=cw) # image weights
325         dataset.indices = random.choices(range(dataset.n), weights=iw, k=dataset.n) # rand weighted idx
326
327     mloss = torch.zeros(3, device=device) # mean losses
328     if RANK != -1:
329         train_loader.sampler.set_epoch(epoch)
330     pbar = enumerate(train_loader)
331     LOGGER.info(f"\n" + "%11s" * 7 % ("Epoch", "GPU_mem", "box_loss", "obj_loss", "cls_loss", "Instances", "Size"))
332     if RANK in {-1, 0}:
333         pbar = tqdm(pbar, total=nb, bar_format=TQDM_BAR_FORMAT) # progress bar
334     optimizer.zero_grad()
335     for i, (imgs, targets, paths, _) in pbar: # batch
336         callbacks.run("on_train_batch_start")
337         ni = i + nb * epoch # number integrated batches (since train start)
338         imgs = imgs.to(device, non_blocking=True).float() / 255 # uint8 to float32, 0-255 to 0.0-1.0
339
340         # Warmup
341         if ni <= nw:
342             xi = [0, nw] # x interp
343             # compute_loss.gr = np.interp(ni, xi, [0.0, 1.0]) # iou loss ratio (obj_loss = 1.0 or iou)
344             accumulate = max(1, np.interp(ni, xi, [1, nbs / batch_size])).round()
345             for j, x in enumerate(optimizer.param_groups):
346                 # bias lr falls from 0.1 to lr0, all other lrs rise from 0.0 to lr0
347                 x["lr"] = np.interp(ni, xi, [hyp["warmup_bias_lr"] if j == 0 else 0.0, x["initial_lr"] * lf(epoch)])
348                 if "momentum" in x:
349                     x["momentum"] = np.interp(ni, xi, [hyp["warmup_momentum"], hyp["momentum"]])
350
351         # Multi-scale
352         if opt.multi_scale:
353             sz = random.randrange(int(imgsz * 0.5), int(imgsz * 1.5) + gs) // gs * gs # size
354             sf = sz / max(imgs.shape[2:]) # scale factor
355             if sf != 1:
356                 ns = [math.ceil(x * sf / gs) * gs for x in imgs.shape[2:]] # new shape (stretched to gs-multiple)
357                 imgs = nn.functional.interpolate(imgs, size=ns, mode="bilinear", align_corners=False)
358
359         # Forward
360         with torch.cuda.amp.autocast(amp):
361             pred = model(imgs) # forward
362             loss, loss_items = compute_loss(pred, targets.to(device)) # loss scaled by batch_size
363             if RANK != -1:
364                 loss *= WORLD_SIZE # gradient averaged between devices in DDP mode
365             if opt.quad:
366                 loss *= 4.0
367
368         # Backward
369         scaler.scale(loss).backward()
370
371         # Optimize - https://pytorch.org/docs/master/notes/amp_examples.html
372         if ni - last_opt_step >= accumulate:
373             scaler.unscale(optimizer) # unscale gradients
374             torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=10.0) # clip gradients
375             scaler.step(optimizer) # optimizer.step
376             scaler.update()
377             optimizer.zero_grad()
378             if ema:
379                 ema.update(model)
380             last_opt_step = ni
381
382         # Log
383         if RANK in {-1, 0}:
384             mloss = (mloss * i + loss_items) / (i + 1) # update mean losses
385             mem = f"{torch.cuda.memory_reserved() / 1e9 if torch.cuda.is_available() else 0:.3g}G" # (GB)
386             pbar.set_description(
387                 ("%11s" * 2 + "%11.4g" * 5)
388                 % (f"{epoch}/{epochs - 1}", mem, *mloss, targets.shape[0], imgs.shape[-1])
389             )
390             callbacks.run("on_train_batch_end", model, ni, imgs, targets, paths, list(mloss))
391         if callbacks.stop_training:
392             return

```

```

393 |         # end batch
394 |
395 |     # Scheduler
396 |     lr = [x["lr"] for x in optimizer.param_groups] # for loggers
397 |     scheduler.step()
398 |
399 |     if RANK in {-1, 0}:
400 |         # mAP
401 |         callbacks.run("on_train_epoch_end", epoch=epoch)
402 |         ema.update_attr(model, include=["yaml", "nc", "hyp", "names", "stride", "class_weights"])
403 |         final_epoch = (epoch + 1 == epochs) or stopper.possible_stop
404 |         if not noval or final_epoch: # Calculate mAP
405 |             results, maps, _ = validate.run(
406 |                 data_dict,
407 |                 batch_size=batch_size // WORLD_SIZE * 2,
408 |                 imgsz=imgsz,
409 |                 half=amp,
410 |                 model=ema.ema,
411 |                 single_cls=single_cls,
412 |                 dataloader=val_loader,
413 |                 save_dir=save_dir,
414 |                 plots=False,
415 |                 callbacks=callbacks,
416 |                 compute_loss=compute_loss,
417 |             )
418 |
419 |         # Update best mAP
420 |         fi = fitness(np.array(results).reshape(1, -1)) # weighted combination of [P, R, mAP@.5, mAP@.5-.95]
421 |         stop = stopper(epoch=epoch, fitness=fi) # early stop check
422 |         if fi > best_fitness:
423 |             best_fitness = fi
424 |         log_vals = list(mloss) + list(results) + lr
425 |
426 |         callbacks.run("on_fit_epoch_end", log_vals, epoch, best_fitness, fi)
427 |
428 |         # Save model
429 |         if (not nosave) or (final_epoch and not evolve): # if save
430 |             ckpt = {
431 |                 "epoch": epoch,
432 |                 "best_fitness": best_fitness,
433 |                 "model": deepcopy(de_parallel(model)).half(),
434 |                 "ema": deepcopy(ema.ema).half(),
435 |                 "updates": ema.updates,
436 |                 "optimizer": optimizer.state_dict(),
437 |                 "opt": vars(opt),
438 |                 "git": GIT_INFO, # {remote, branch, commit} if a git repo
439 |                 "date": datetime.now().isoformat(),
440 |             }
441 |
442 |             # Save last, best and delete
443 |             torch.save(ckpt, last)
444 |             if best_fitness == fi:
445 |                 torch.save(ckpt, best)
446 |             if opt.save_period > 0 and epoch % opt.save_period == 0:
447 |                 torch.save(ckpt, w / f"epoch{epoch}.pt")
448 |             del ckpt
449 |             callbacks.run("on_model_save", last, epoch, final_epoch, best_fitness, fi)
450 |
451 |         # EarlyStopping
452 |         if RANK != -1: # if DDP training
453 |             broadcast_list = [stop if RANK == 0 else None]
454 |             dist.broadcast_object_list(broadcast_list, 0) # broadcast 'stop' to all ranks
455 |             if RANK != 0:
456 |                 stop = broadcast_list[0]
457 |             if stop:
458 |                 break # must break all DDP ranks
459 |
460 |     # end epoch
461 | # end training
462 | if RANK in {-1, 0}:
463 |     LOGGER.info(f"\n(epoch - start_epoch + 1) epochs completed in {(time.time() - t0) / 3600:.3f} hours.")
464 |     for f in last, best:
465 |         if f.exists():
466 |             strip_optimizer(f) # strip optimizers
467 |             if f is best:
468 |                 LOGGER.info(f"\nValidating {f}...")
469 |                 results, _, _ = validate.run(
470 |                     data_dict,
471 |                     batch_size=batch_size // WORLD_SIZE * 2,
472 |                     imgsz=imgsz,
473 |                     model=attempt_load(f, device).half(),
474 |                     iou_thres=0.65 if is_coco else 0.60, # best pycocotools at iou 0.65
475 |                     single_cls=single_cls,
476 |                     dataloader=val_loader,
477 |                     save_dir=save_dir,
478 |                     save_json=is_coco,
479 |                     verbose=True,
480 |                     plots=plots,
481 |                     callbacks=callbacks,
482 |                     compute_loss=compute_loss,
483 |                 ) # val best model with plots
484 |                 if is_coco:
485 |                     callbacks.run("on_fit_epoch_end", list(mloss) + list(results) + lr, epoch, best_fitness, fi)
486 |
487 |             callbacks.run("on_train_end", last, best, epoch, results)
488 |
489 | torch.cuda.empty_cache()
490 | return results
491 |
492 | def parse_opt(known=False):
493 |

```

```

494 parser = argparse.ArgumentParser()
495 parser.add_argument("--weights", type=str, default=ROOT / "yolov5s.pt", help="initial weights path")
496 parser.add_argument("--cfg", type=str, default="", help="model.yaml path")
497 parser.add_argument("--data", type=str, default=ROOT / "data/coco128.yaml", help="dataset.yaml path")
498 parser.add_argument("--hyp", type=str, default=ROOT / "data/hyps/hyp.scratch-low.yaml", help="hyperparameters path")
499 parser.add_argument("--epochs", type=int, default=100, help="total training epochs")
500 parser.add_argument("--batch-size", type=int, default=16, help="total batch size for all GPUs, -1 for autobatch")
501 parser.add_argument("--imgsz", "--img", "--img-size", type=int, default=640, help="train, val image size (pixels)")
502 parser.add_argument("--rect", action="store_true", help="rectangular training")
503 parser.add_argument("--resume", nargs="?", const=True, default=False, help="resume most recent training")
504 parser.add_argument("--nosave", action="store_true", help="only save final checkpoint")
505 parser.add_argument("--noval", action="store_true", help="only validate final epoch")
506 parser.add_argument("--noautoanchor", action="store_true", help="disable AutoAnchor")
507 parser.add_argument("--noplots", action="store_true", help="save no plot files")
508 parser.add_argument("--evolve", type=int, nargs="?", const=300, help="evolve hyperparameters for x generations")
509 parser.add_argument(
510 |   "--evolve_population", type=str, default=ROOT / "data/hyps", help="location for loading population"
511 | )
512 parser.add_argument("--resume_evolve", type=str, default=None, help="resume evolve from last generation")
513 parser.add_argument("--bucket", type=str, default="", help="gsutil bucket")
514 parser.add_argument("--cache", type=str, nargs="?", const="ram", help="image --cache ram/disk")
515 parser.add_argument("--image-weights", action="store_true", help="use weighted image selection for training")
516 parser.add_argument("--device", default="", help="cuda device, i.e. 0 or 0,1,2,3 or cpu")
517 parser.add_argument("--multi-scale", action="store_true", help="vary img-size +/- 50%%")
518 parser.add_argument("--single-cls", action="store_true", help="train multi-class data as single-class")
519 parser.add_argument("--optimizer", type=str, choices=["SGD", "Adam", "AdamW"], default="SGD", help="optimizer")
520 parser.add_argument("--sync-bn", action="store_true", help="use SyncBatchNorm, only available in DDP mode")
521 parser.add_argument("--workers", type=int, default=8, help="max dataloader workers (per RANK in DDP mode)")
522 parser.add_argument("--project", default=ROOT / "runs/train", help="save to project/name")
523 parser.add_argument("--name", default="exp", help="save to project/name")
524 parser.add_argument("--exist-ok", action="store_true", help="existing project/name ok, do not increment")
525 parser.add_argument("--quad", action="store_true", help="quad dataloader")
526 parser.add_argument("--cos-lr", action="store_true", help="cosine LR scheduler")
527 parser.add_argument("--label-smoothing", type=float, default=0.0, help="Label smoothing epsilon")
528 parser.add_argument("--patience", type=int, default=100, help="EarlyStopping patience (epochs without improvement)")
529 parser.add_argument("--freeze", nargs="+", type=int, default=[0], help="Freeze layers: backbone=10, first3=0 1 2")
530 parser.add_argument("--save-period", type=int, default=-1, help="Save checkpoint every x epochs (disabled if < 1)")
531 parser.add_argument("--seed", type=int, default=0, help="Global training seed")
532 parser.add_argument("--local_rank", type=int, default=-1, help="Automatic DDP Multi-GPU argument, do not modify")
533
534 # Logger arguments
535 parser.add_argument("--entity", default=None, help="Entity")
536 parser.add_argument("--upload_dataset", nargs="?", const=True, default=False, help='Upload data, "val" option')
537 parser.add_argument("--bbox_interval", type=int, default=-1, help="Set bounding-box image logging interval")
538 parser.add_argument("--artifact_alias", type=str, default="latest", help="Version of dataset artifact to use")
539
540 # NDJSON logging
541 parser.add_argument("--ndjson-console", action="store_true", help="Log ndjson to console")
542 parser.add_argument("--ndjson-file", action="store_true", help="Log ndjson to file")
543
544 return parser.parse_known_args()[0] if known else parser.parse_args()
545
546
547 def main(opt, callbacks=Callbacks()):
548
549     if RANK in {-1, 0}:
550         print_args(vars(opt))
551         check_git_status()
552         check_requirements(ROOT / "requirements.txt")
553
554     # Resume (from specified or most recent last.pt)
555     if opt.resume and not check_comet_resume(opt) and not opt.evolve:
556         last = Path(check_file(opt.resume)) if isinstance(opt.resume, str) else get_latest_run()
557         opt_yaml = last.parent.parent / "opt.yaml" # train options yaml
558         opt_data = opt.data # original dataset
559         if opt_yaml.is_file():
560             with open(opt_yaml, errors="ignore") as f:
561                 d = yaml.safe_load(f)
562         else:
563             d = torch.load(last, map_location="cpu")["opt"]
564         opt = argparse.Namespace(**d) # replace
565         opt.cfg, opt.weights, opt.resume = "", str(last), True # reinstate
566         if is_url(opt.data):
567             opt.data = check_file(opt.data) # avoid HUB resume auth timeout
568         else:
569             opt.data, opt.cfg, opt.hyp, opt.weights, opt.project = (
570                 check_file(opt.data),
571                 check_yaml(opt.cfg),
572                 check_yaml(opt.hyp),
573                 str(opt.weights),
574                 str(opt.project),
575             ) # checks
576         assert len(opt.cfg) or len(opt.weights), "either --cfg or --weights must be specified"
577         if opt.evolve:
578             if opt.project == str(ROOT / "runs/train"): # if default project name, rename to runs/evolve
579                 opt.project = str(ROOT / "runs/evolve")
580             opt.exist_ok, opt.resume = opt.resume, False # pass resume to exist_ok and disable resume
581         if opt.name == "cfg":
582             opt.name = Path(opt.cfg).stem # use model.yaml as name
583         opt.save_dir = str(increment_path(Path(opt.project) / opt.name, exist_ok=opt.exist_ok))
584
585     # DDP mode
586     device = select_device(opt.device, batch_size=opt.batch_size)
587     if LOCAL_RANK != -1:
588         msg = "is not compatible with YOLOv5 Multi-GPU DDP training"
589         assert not opt.image_weights, f"--image-weights {msg}"
590         assert not opt.evolve, f"--evolve {msg}"
591         assert opt.batch_size != -1, f"AutoBatch with --batch-size -1 {msg}, please pass a valid --batch-size"
592         assert opt.batch_size % WORLD_SIZE == 0, f"--batch-size {opt.batch_size} must be multiple of WORLD_SIZE"
593         assert torch.cuda.device_count() > LOCAL_RANK, "insufficient CUDA devices for DDP command"
594         torch.cuda.set_device(LOCAL_RANK)
595         device = torch.device("cuda", LOCAL_RANK)
596         dist.init_process_group(
597             backend="nccl" if dist.is_nccl_available() else "gloo", timeout=timedelta(seconds=10800)
598         )
599
600     # Train

```

```

601 if not opt.evolve:
602 |   train(opt.hyp, opt, device, callbacks)
603
604 # Evolve hyperparameters (optional)
605 else:
606 |   # Hyperparameter evolution metadata (including this hyperparameter True-False, lower_limit, upper_limit)
607 |   meta = {
608 |       "lr0": (False, 1e-5, 1e-1), # initial learning rate (SGD=1E-2, Adam=1E-3)
609 |       "lrf": (False, 0.01, 1.0), # final OneCycleLR learning rate (lr0 * lrf)
610 |       "momentum": (False, 0.6, 0.98), # SGD momentum/Adam beta1
611 |       "weight_decay": (False, 0.0, 0.001), # optimizer weight decay
612 |       "warmup_epochs": (False, 0.0, 5.0), # warmup epochs (fractions ok)
613 |       "warmup_momentum": (False, 0.0, 0.95), # warmup initial momentum
614 |       "warmup_bias_lr": (False, 0.0, 0.2), # warmup initial bias lr
615 |       "box": (False, 0.02, 0.2), # box loss gain
616 |       "cls": (False, 0.2, 4.0), # cls loss gain
617 |       "cls_pw": (False, 0.5, 2.0), # cls BCELoss positive_weight
618 |       "obj": (False, 0.2, 4.0), # obj loss gain (scale with pixels)
619 |       "obj_pw": (False, 0.5, 2.0), # obj BCELoss positive_weight
620 |       "iou_t": (False, 0.1, 0.7), # IoU training threshold
621 |       "anchor_t": (False, 2.0, 8.0), # anchor-multiple threshold
622 |       "anchors": (False, 2.0, 10.0), # anchors per output grid (0 to ignore)
623 |       "fl_gamma": (False, 0.0, 2.0), # focal loss gamma (efficientDet default gamma=1.5)
624 |       "hsv_h": (True, 0.0, 0.1), # image HSV-Hue augmentation (fraction)
625 |       "hsv_s": (True, 0.0, 0.9), # image HSV-Saturation augmentation (fraction)
626 |       "hsv_v": (True, 0.0, 0.9), # image HSV-Value augmentation (fraction)
627 |       "degrees": (True, 0.0, 45.0), # image rotation (+/- deg)
628 |       "translate": (True, 0.0, 0.9), # image translation (+/- fraction)
629 |       "scale": (True, 0.0, 0.9), # image scale (+/- gain)
630 |       "shear": (True, 0.0, 10.0), # image shear (+/- deg)
631 |       "perspective": (True, 0.0, 0.001), # image perspective (+/- fraction), range 0-0.001
632 |       "flipud": (True, 0.0, 1.0), # image flip up-down (probability)
633 |       "fliplr": (True, 0.0, 1.0), # image flip left-right (probability)
634 |       "mosaic": (True, 0.0, 1.0), # image mosaic (probability)
635 |       "mixup": (True, 0.0, 1.0), # image mixup (probability)
636 |       "copy_paste": (True, 0.0, 1.0), # segment copy-paste (probability)
637 |   }
638
639 # GA configs
640 pop_size = 50
641 mutation_rate_min = 0.01
642 mutation_rate_max = 0.5
643 crossover_rate_min = 0.5
644 crossover_rate_max = 1
645 min_elite_size = 2
646 max_elite_size = 5
647 tournament_size_min = 2
648 tournament_size_max = 10
649
650 with open(opt.hyp, errors="ignore") as f:
651 |   hyp = yaml.safe_load(f) # load hyps dict
652 |   if "anchors" not in hyp: # anchors commented in hyp.yaml
653 |       |   hyp["anchors"] = 3
654 |   if opt.noautoanchor:
655 |       |   del hyp["anchors"], meta["anchors"]
656 |   opt.noval, opt.nosave, save_dir = True, True, Path(opt.save_dir) # only val/save final epoch
657 |   ei = [isinstance(x, (int, float)) for x in hyp.values()] # evolvable indices
658 |   evolve_yaml, evolve_csv = save_dir / "hyp_evolve.yaml", save_dir / "evolve.csv"
659 |   if opt.bucket:
660 |       |   # download evolve.csv if exists
661 |       |   subprocess.run(
662 |       |       [
663 |       |           "gsutil",
664 |       |           "cp",
665 |       |           f"gs://{opt.bucket}/evolve.csv",
666 |       |           str(evolve_csv),
667 |       |       ]
668 |       )
669
670 # Delete the items in meta dictionary whose first value is False
671 del_ = [item for item, value in meta.items() if value[0] is False]
672 hyp_GA = hyp.copy() # Make a copy of hyp dictionary
673 for item in del_:
674 |   del meta[item] # Remove the item from meta dictionary
675 |   del hyp_GA[item] # Remove the item from hyp_GA dictionary
676
677 # Set lower_limit and upper_limit arrays to hold the search space boundaries
678 lower_limit = np.array([meta[k][1] for k in hyp_GA.keys()])
679 upper_limit = np.array([meta[k][2] for k in hyp_GA.keys()])
680
681 # Create gene_ranges list to hold the range of values for each gene in the population
682 gene_ranges = [(lower_limit[i], upper_limit[i]) for i in range(len(upper_limit))]
683
684 # Initialize the population with initial_values or random values
685 initial_values = []
686
687 # If resuming evolution from a previous checkpoint
688 if opt.resume_evolve is not None:
689 |   assert os.path.isfile(ROOT / opt.resume_evolve), "evolve population path is wrong!"
690 |   with open(ROOT / opt.resume_evolve, errors="ignore") as f:
691 |       |   evolve_population = yaml.safe_load(f)
692 |       |   for value in evolve_population.values():
693 |       |       |   value = np.array([value[k] for k in hyp_GA.keys()])
694 |       |       |   initial_values.append(list(value))
695
696 # If not resuming from a previous checkpoint, generate initial values from .yaml files in opt.evolve_population
697 else:
698 |   yaml_files = [f for f in os.listdir(opt.evolve_population) if f.endswith(".yaml")]
699 |   for file_name in yaml_files:
700 |       |   with open(os.path.join(opt.evolve_population, file_name)) as yaml_file:
701 |       |       |   value = yaml.safe_load(yaml_file)
702 |       |       |   value = np.array([value[k] for k in hyp_GA.keys()])
703 |       |       |   initial_values.append(list(value))
704
705 # Generate random values within the search space for the rest of the population

```

```

706 if initial_values is None:
707     population = [generate_individual(gene_ranges, len(hyp_GA)) for _ in range(pop_size)]
708 elif pop_size > 1:
709     population = [generate_individual(gene_ranges, len(hyp_GA)) for _ in range(pop_size - len(initial_values))]
710     for initial_value in initial_values:
711         population = [initial_value] + population
712
713 # Run the genetic algorithm for a fixed number of generations
714 list_keys = list(hyp_GA.keys())
715 for generation in range(opt.evolve):
716     if generation >= 1:
717         save_dict = {}
718         for i in range(len(population)):
719             little_dict = {list_keys[j]: float(population[i][j]) for j in range(len(population[i]))}
720             save_dict[f"gen{str(generation)}number{str(i)}"] = little_dict
721
722         with open(save_dir / "evolve_population.yaml", "w") as outfile:
723             yaml.dump(save_dict, outfile, default_flow_style=False)
724
725 # Adaptive elite size
726 elite_size = min_elite_size + int((max_elite_size - min_elite_size) * (generation / opt.evolve))
727 # Evaluate the fitness of each individual in the population
728 fitness_scores = []
729 for individual in population:
730     for key, value in zip(hyp_GA.keys(), individual):
731         hyp_GA[key] = value
732     hyp.update(hyp_GA)
733     results = train(hyp.copy(), opt, device, callbacks)
734     callbacks = Callbacks()
735     # Write mutation results
736     keys = (
737         "metrics/precision",
738         "metrics/recall",
739         "metrics/mAP_0.5",
740         "metrics/mAP_0.5:0.95",
741         "val/box_loss",
742         "val/obj_loss",
743         "val/cls_loss",
744     )
745     print_mutation(keys, results, hyp.copy(), save_dir, opt.bucket)
746     fitness_scores.append(results[2])
747
748 # Select the fittest individuals for reproduction using adaptive tournament selection
749 selected_indices = []
750 for _ in range(pop_size - elite_size):
751     # Adaptive tournament size
752     tournament_size = max(
753         2, tournament_size_min,
754         int(min(tournament_size_max, pop_size) - (generation / (opt.evolve / 10))),
755     )
756     # Perform tournament selection to choose the best individual
757     tournament_indices = random.sample(range(pop_size), tournament_size)
758     tournament_fitness = [fitness_scores[j] for j in tournament_indices]
759     winner_index = tournament_indices[tournament_fitness.index(max(tournament_fitness))]
760     selected_indices.append(winner_index)
761
762 # Add the elite individuals to the selected indices
763 elite_indices = [i for i in range(pop_size) if fitness_scores[i] in sorted(fitness_scores)[-elite_size:]]
764 selected_indices.extend(elite_indices)
765 # Create the next generation through crossover and mutation
766 next_generation = []
767 for _ in range(pop_size):
768     parent1_index = selected_indices[random.randint(0, pop_size - 1)]
769     parent2_index = selected_indices[random.randint(0, pop_size - 1)]
770     # Adaptive crossover rate
771     crossover_rate = max(
772         crossover_rate_min, min(crossover_rate_max, crossover_rate_max - (generation / opt.evolve))
773     )
774     if random.uniform(0, 1) < crossover_rate:
775         crossover_point = random.randint(1, len(hyp_GA) - 1)
776         child = population[parent1_index][:crossover_point] + population[parent2_index][crossover_point:]
777     else:
778         child = population[parent1_index]
779     # Adaptive mutation rate
780     mutation_rate = max(
781         mutation_rate_min, min(mutation_rate_max, mutation_rate_max - (generation / opt.evolve))
782     )
783     for j in range(len(hyp_GA)):
784         if random.uniform(0, 1) < mutation_rate:
785             child[j] += random.uniform(-0.1, 0.1)
786             child[j] = min(max(child[j], gene_ranges[j][0]), gene_ranges[j][1])
787     next_generation.append(child)
788 # Replace the old population with the new generation
789 population = next_generation
790 # Print the best solution found
791 best_index = fitness_scores.index(max(fitness_scores))
792 best_individual = population[best_index]
793 print("Best solution found:", best_individual)
794 # Plot results
795 plot_evolve(evolve_csv)
796 LOGGER.info(
797     f"Hyperparameter evolution finished {opt.evolve} generations\n"
798     f"Results saved to {colorstr('bold', save_dir)}\n"
799     f"Usage example: $ python train.py --hyp {evolve_yaml}"
800 )
801
802
803 def generate_individual(input_ranges, individual_length):
804     individual = []
805     for i in range(individual_length):
806         lower_bound, upper_bound = input_ranges[i]
807         individual.append(random.uniform(lower_bound, upper_bound))
808     return individual
809
810
811 def run(**kwargs):
812     opt = parse_opt(True)
813     for k, v in kwargs.items():
814         setattr(opt, k, v)
815     main(opt)
816     return opt
817
818
819 if __name__ == "__main__":
820     opt = parse_opt()
821     main(opt)

```

The model was trained using the following command:

```
python yolov5/train.py --img 640 --batch 4 --epochs 50 --data helmet-detection/data.yaml --weights yolov5s.pt --name helmet_detect_final
```

Key parameters:

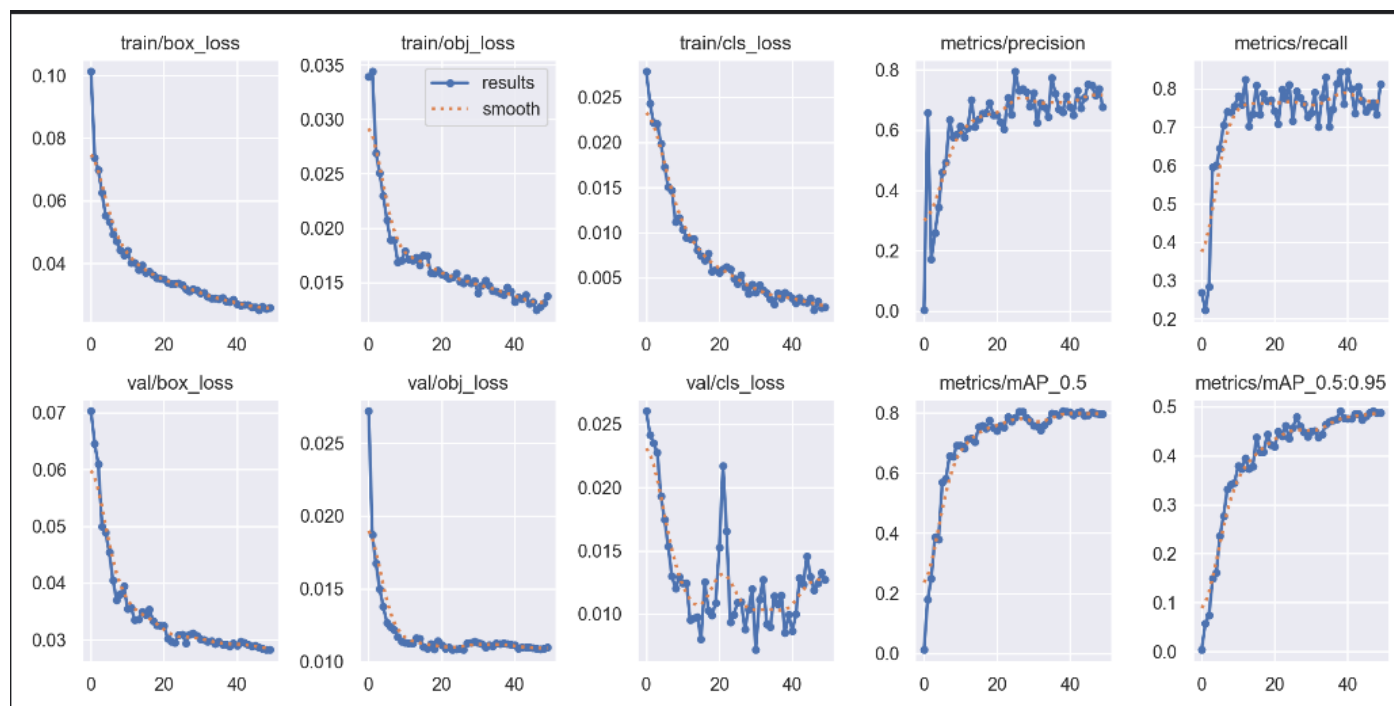
- **Image size:** 640×640 pixels
- **Batch size:** 4
- **Epochs:** 50
- **Weights:** yolov5s.pt (pretrained on COCO)

After 50 epochs, the training results including loss plots, precision-recall curves, and confusion matrices were saved in:

```
C:\Users\hplap\PycharmProjects\pythonProject1\yolov5\runs\train\helmet_detect_final
```

This directory included:

- weights/best.pt: the best model based on validation performance
- results.png: training metrics visualized



MODEL EVALUATION

After training, the model was evaluated on unseen test data to assess its performance. I used YOLOv5's detection script to perform inference:

Command used for testing the model:

```
python yolov5/detect.py --weights yolov5/runs/train/helmet_detect_test3/weights/best.pt --img 640 -  
-conf 0.25 --source "helmet-detection/test/images"
```

This command produced:

- Detection results saved under yolov5/runs/detect/exp
- Images with bounding boxes labeled “helmet” or “no-helmet”
- Confidence scores displayed on the images

Observations:

- ✓ The model accurately detected helmets and non-helmet cases across varied lighting conditions and backgrounds.
- ✓ However, performance depended on the diversity of training images.

REAL-TIME WEBCAME DETECTION

To test real-time detection, I used my laptop webcam as the input source:

```
python yolov5/detect.py --weights yolov5/runs/train/helmet_detect_final/weights/best.pt --img 640 --conf 0.25 --source 0 --view-img
```

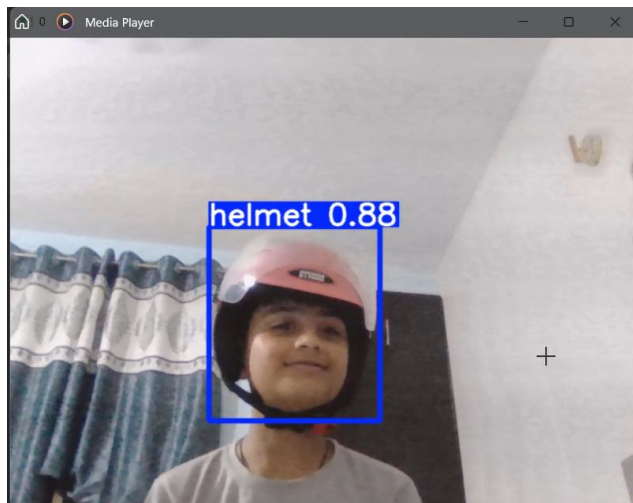
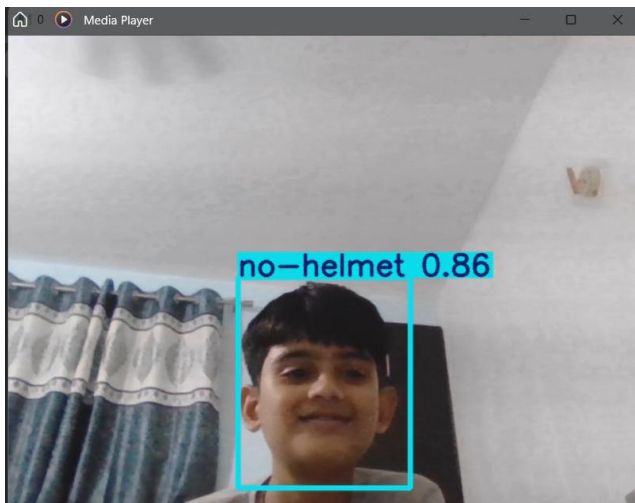
Challenges Faced:

- Model misclassified at first due to insufficient training.
- Close-up images improved detection significantly.
- Increasing training epochs from 3 to 50 resolved earlier inconsistencies.

Results:

- Detected “helmet” and “no-helmet” with reasonable accuracy.
- Bounding boxes and labels updated in real time.

Output:



STREAMLIT WEB APP INTEGRATION:

To make detection accessible, I built a Streamlit web app for browser-based interaction.

File Structure:

helmet_app/

- app.py
- detect_utils.py
- test_images/
- predictions/
- yolov5/

Main Components:

- app.py: Handles UI, file uploads, and displays results.
- detect_utils.py: Loads the trained model and runs inference.

app.py:

```
C: > Users > hplap > PycharmProjects > pythonProject1 > helmet_app > app.py
1 import streamlit as st
2 from detect_utils import run_detection
3 import os
4
5 st.set_page_config(page_title="Helmet Detection Web App", layout="centered")
6 st.title("Helmet Detection using YOLOv5")
7
8 # Ensure folders exist
9 os.makedirs("test_images", exist_ok=True)
10 os.makedirs("predictions", exist_ok=True)
11
12 uploaded_file = st.file_uploader("Upload an image", type=["jpg", "png", "jpeg"])
13
14 if uploaded_file is not None:
15     image_path = os.path.join("test_images", uploaded_file.name)
16     with open(image_path, "wb") as f:
17         f.write(uploaded_file.read())
18
19     st.image(image_path, caption="Uploaded Image", use_column_width=True)
20
21     if st.button("Detect Helmet"):
22         pred_image_path, results = run_detection(image_path)
23
24         if os.path.exists(pred_image_path):
25             st.image(pred_image_path, caption="Detected Result", use_column_width=True)
26             st.success("Detection Complete ")
27         else:
28             st.error("Something went wrong: result image not found.")
```

detect_utils.py:

```
C: > Users > hplap > PycharmProjects > pythonProject1 > helmet_app > detect_utils.py
1 import torch
2 import cv2
3 import os
4 from pathlib import Path
5 import uuid
6
7 # Load model once
8 model = torch.hub.load(
9     "yolov5", # or path to your cloned repo
10     "custom",
11     path="yolov5/runs/train/helmet_detect_final/weights/best.pt",
12     source="local"
13 )
14 model.conf = 0.25 # confidence threshold
15 model.names = ['helmet', 'no-helmet']
16 def run_detection(image_path):
17     # Read and run detection
18     results = model(image_path)
19
20     # Generate unique filename
21     save_dir = Path("predictions")
22     save_dir.mkdir(parents=True, exist_ok=True)
23     unique_name = f"{uuid.uuid4().hex}.jpg"
24     result_path = save_dir / unique_name
25
26     # Render prediction result and save manually
27     rendered_img = results.render()[0] # returns list; we take the first image
28     cv2.imwrite(str(result_path), rendered_img)
29     return str(result_path), results
```

How it works:

1. User uploads an image.
2. Streamlit displays the image.
3. On clicking "Detect Helmet", the image is passed to the model.
4. Output image is saved and shown with detections.

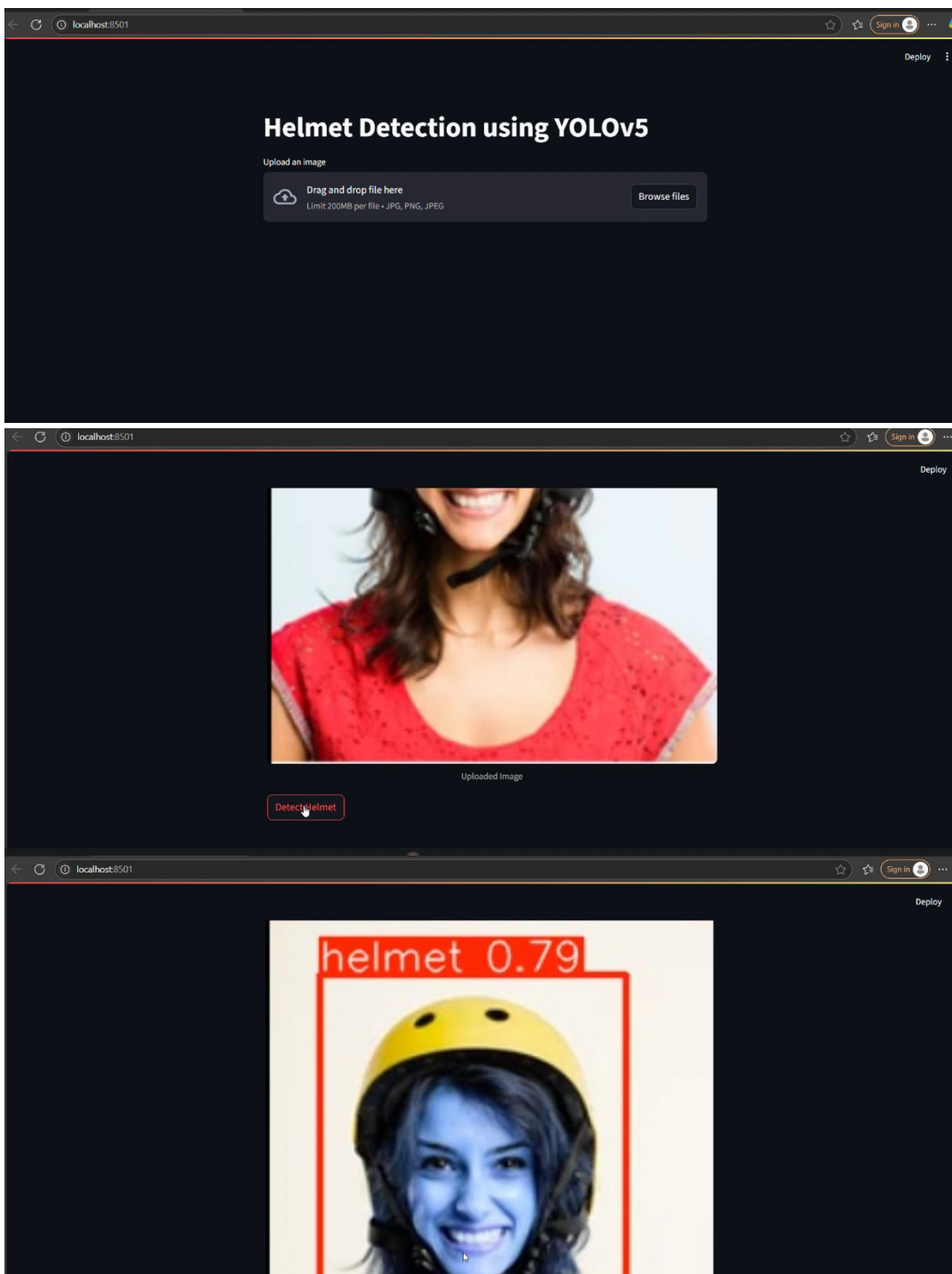
Output Saved To:

- predictions/ directory, dynamically generated using UUID.

Command to run the app:

```
streamlit run app.py
```

Output:



RESULTS AND OBSERVATIONS

The trained YOLOv5 model produced promising results after 50 epochs of training. The accuracy of helmet detection improved significantly compared to earlier short training runs.

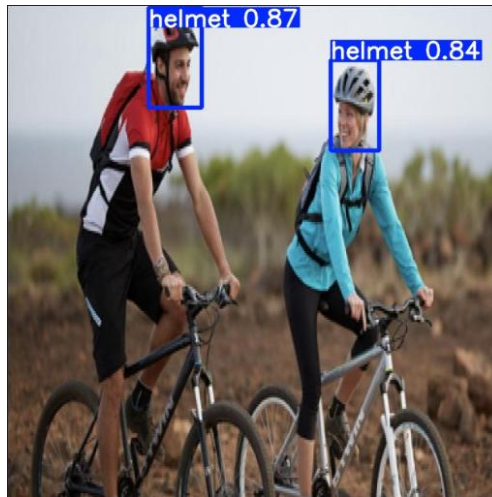
Key Results:

- The model accurately detected both helmet and no-helmet labels in test images.
- Performance was robust across a variety of backgrounds and lighting conditions.
- Detection confidence improved with the number of epochs and diversity in the dataset.

Metrics:

- Precision, recall, and mAP (mean Average Precision) were visualized in results.png:
 - **Precision:** High confidence for positive detections.
 - **Recall:** Improved significantly after full training.
 - **mAP@0.5:** Reached a satisfactory level for a 2-class problem.

Outputs:



CHALLENGES AND SOLUTIONS

The development process involved several practical and technical challenges:

Challenges:

- Initial misclassifications
- Low variations in dataset
- Webcam showing wrong results
- Script execution blocked in powershell
- Streamlit failed to locate prediction images
- Git errors during training

Solutions:

- ✓ Augmented the dataset and included diverse samples
- ✓ Re-trained model and ensured clean label annotations
- ✓ Used Set-ExecutionPolicy Bypass to activate the virtual environment
- ✓ Fixed path handling and ensured directory creation before saving images
- ✓ Installed Git and added it to the system PATH, or ignored version tracking

These obstacles were common when working with deep learning and deployment tools, and overcoming them added resilience and polish to the final project.

CONCLUSION AND FUTURE WORK

This project successfully implemented a deep learning-based helmet detection system using YOLOv5. From data annotation to real-time deployment through a Streamlit web interface, every stage demonstrated practical application of object detection in safety monitoring.

Conclusion Highlights:

- A custom YOLOv5 model was trained on a labeled dataset of helmet and no-helmet images.
- The model achieved real-time inference through both command-line and browser-based interfaces.
- The Streamlit app offered a user-friendly platform for testing the model with new images.

Future Enhancements:

- **Dataset Expansion:** Include more samples with various helmet types and conditions.
- **License Plate Integration:** Combine helmet detection with vehicle number plate recognition for automatic enforcement.
- **Mobile Deployment:** Convert the model using TensorFlow Lite or ONNX for mobile applications.
- **Multi-Camera Support:** Expand to process multiple video feeds from surveillance systems.

This project stands as a valuable tool for enhancing road safety using AI.

REFERENCES

1. **YOLOv5 GitHub Repository:**
<https://github.com/ultralytics/yolov5>
2. **Labellmg (Image Annotation Tool):**
<https://github.com/tzutalin/labellmg>
3. **PyTorch Documentation:**
<https://pytorch.org/docs/stable/index.html>
4. **Streamlit Documentation:**
<https://docs.streamlit.io>
5. **Custom Object Detection with YOLOv5 Tutorial:**
Various tutorials on YouTube and Medium were consulted to understand custom training and deployment.